

Day 2: Validate Password

1. Day Number

Day 2 of the Python foundation plan

2. Topic Name

if-else, comparison operator, and Boolean result

Today you will learn how to:

- Compare two strings using `==`
 - Think about conditions as `True` or `False`
 - Run different code with `if` and `else`
 - Recognize that password comparison is case-sensitive
-

3. Connection with Yesterday

On Day 1, you asked the user to create a password and stored it in a variable.

Today, you will store a password and ask the user to enter it again. The program will compare both values and decide whether login should succeed.

This changes the program from simply remembering data to making a decision with that data.

4. Important Topics

Variables

Two variables are needed:

- One for the saved password
- One for the password entered during login

Strings

Passwords are stored as strings because they may contain letters, numbers, spaces, and symbols.

Comparison operator `==`

`==` asks whether two values are equal.

```
"blue" == "blue"
```

This comparison produces `True`.

```
"blue" == "Blue"
```

This comparison produces `False`.

Boolean thinking

A Boolean value has only two possibilities:

```
True
False
```

A password comparison is either true or false.

if-else

if runs a block when a condition is true. else runs the alternative block when it is false.

```
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

5. Foundational Notes

Assignment and comparison are different

Use one equals sign to assign a value:

```
status = "active"
```

Use two equals signs to compare values:

```
status == "active"
```

A condition must produce a result

Python evaluates the expression after if.

```
saved_color == entered_color
```

The result is either True or False.

Indentation controls each branch

Statements belonging to if or else must be indented:

```
if condition:
    print("Condition is true")
else:
    print("Condition is false")
```

Password matching is case-sensitive

Python treats uppercase and lowercase characters as different:

```
Python123
python123
```

These two passwords do not match.

Spaces are part of a password

An extra space at the beginning or end changes the string. Do not automatically clean a password with `strip()` because spaces might be intentional.

Only one branch should run

For a single login attempt:

- A correct password should print the success message.
- An incorrect password should print the wrong-password message.

Both messages should not appear together.

6. Easy Example

Compare a saved color with an entered color:

```
saved_color = "blue"
entered_color = input("Enter the saved color: ")

if entered_color == saved_color:
    print("Colors match.")
else:
    print("Colors do not match.")
```

This shows comparison and `if-else` using harmless data. Apply the same idea to today's password problem without copying this example word for word.

7. Problem Statement

Create a beginner-friendly Python program that:

1. Stores a password in a variable.
2. Asks the user to enter the password again.
3. Compares the entered password with the stored password.
4. Prints "Login successful" when they match.
5. Prints "Wrong password" when they do not match.

Use a functional approach and allow only one login attempt.

8. Concepts Used

- Variables
- Strings
- User input
- Assignment operator =
- Comparison operator ==
- Boolean result
- `if-else`
- Indentation
- Function

- Output
-

9. Thought Process

Step 1: Store the correct password

The program needs a known password to compare against.

Step 2: Ask for a login password

Use `input()` to receive the user's attempt and store it separately.

Step 3: Build the condition

Compare the entered value with the saved value using `==`.

Step 4: Handle a match

If the condition is true, display the login-success message.

Step 5: Handle a mismatch

Otherwise, display the wrong-password message.

Step 6: Keep the decision inside one function

Define a function containing the stored password, input, comparison, and output. Call it once.

10. Pseudocode

```
START

DEFINE a login function
    STORE the correct password
    ASK the user to enter the password

    IF entered password is equal to stored password
        DISPLAY "Login successful"
    ELSE
        DISPLAY "Wrong password"

CALL the login function

END
```

11. Suggested Solving Approach: Functional Approach

Create one function for the login check.

Inside it:

1. Create the stored-password variable.
2. Receive the entered password.
3. Use one `if-else` decision.
4. Print the appropriate result.

The comparison pattern is:

```
if first_value == second_value:
    # Matching-value action
else:
    # Different-value action
```

Replace the general names and comments with the password logic.

12. Easy Edge Cases

Wrong password

If even one character is different, the values should not match.

Empty input

Pressing Enter without typing creates an empty string. It will be rejected unless the stored password is also empty.

Uppercase and lowercase difference

If the saved password is:

Hello123

Then this input should be treated as different:

hello123

Extra space

This input contains one additional character and therefore should not match:

Hello123

13. Expected Input

One password attempt.

Correct example:

Enter password: bank123

Incorrect example:

Enter password: bank456

14. Expected Output

When the password matches:

```
Login successful
```

When the password does not match:

```
Wrong password
```

15. Hint Only

You need two password variables and this decision shape:

```
if entered_value == saved_value:  
    # Print success  
else:  
    # Print failure
```

Remember that = stores a value, while == compares two values.

Do not copy a complete solution. Build it from the pseudocode.